piRover Builds with K2

piRover – User Blink Rev 1.3

Overview:

In this activity you learn to use a variable to hold a value. You will set a value for the variable using the Python input function. You'll then use an IF statement to test the value of the variable. Based on the state of the variable you will change the state of the piRover's LEDs.

The requirements for the solution are listed below.

- The user will see a welcome message indicating that this is the User Inputs and Decision activity.
- The user will be prompted for which LED to blink. Valid values will be red, green, and blue. The user will be notified if an invalid value is entered.
- The user will be prompted for the blink speed, entering either fast or slow. The user will be notified if an invalid value is entered.
- The solution will blink the LEDs using the color and relative speed specified.

Prerequisites:

Prior to beginning the instruction provided in this lesson you must have completed the following:

1. piRover Blink

Performance Outcomes:

1. Use the print() function to inform the user
2. Use the input() function to prompt the user for a value
3. Create a variable and set the value using the input() function.
4. Test the value of a variable using the IF/ELIF/ELSE structure and control which segment of code is executed.
5. Use variables as arguments in functions such as sleep()
6. Review GPIO code from Blink activity

Resources:

1. Variables in Python
2. If statements in Python
3. Input function in Python

Materials:

1. piRover

© 2020, Keith E. Kelly. All rights reserved.

piRover Builds with K2

Part 1 – Set Up

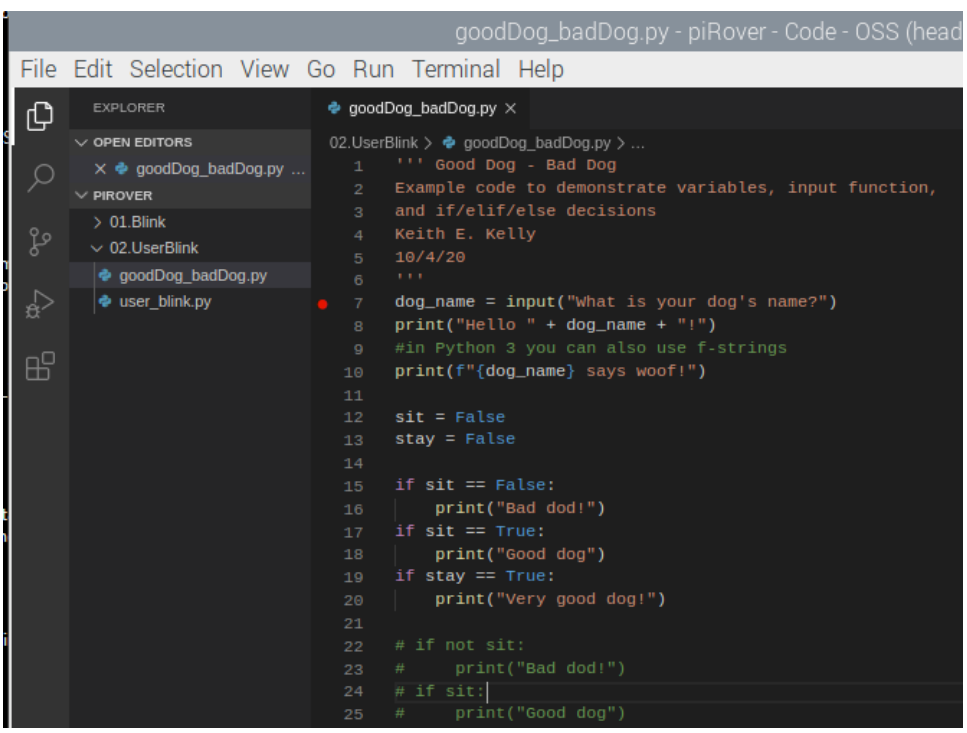
1. Preview the overview of variables on Variables in Python resource.
2. Preview the concepts of decisions using if statement using this If statements in Python resource.
3. Preview the input() function using the Input function in Python resource.
4. Prepare your workspace for this activity.
    a. Connect to your piRover using VNC and open VS Code
    b. Create a directory for this week's work if this was not in the prior session
    c. Using the VS Code terminal window, change to this week's directory.
    d. Download the starter files for the activity using the wget instructions below.

wget https://k2controls.github.io/piRover1/lessons/24/goodDog_badDog.py

wget https://k2controls.github.io/piRover1/lessons/24/user_blink.py

Part 2 – Introducing Variables, input(), and decisions

5. Open the goodDog_badDog.py file in the VS Code editor. Set a breakpoint at line 7. Press F5 and follow along with the instructor as the concepts and use of variables, input, and decisions are explained.



© 2020, Keith E. Kelly. All rights reserved.

6. Here is a list of take-aways from goodDog_badDog example.
    a. dog_name is a variable. It is a container for the name entered by the user.
    b. dog_name is lower case and uses and underscore to separate words. This is a standard in the Python language.
    c. dog_name holds a series of characters returned from the keyboard via the input() function. This is variable is known as having a "string" data type (string of characters).
    d. The input() function returns the string of characters that a user types until she or he presses the Enter key.
    e. Function parameters are values that are provided to the function and located inside the parentheses that follows. The input function accepts a string parameter prompting the user for what value to enter, in this case "What is your dog's name?"
    f. The quotation marks are used around the string parameter on line 7 because this is literally what you want displayed. Later you might include a variable here.
    g. The print() function is used to display text in the terminal window. In line 8 a string value is provided.
    h. String values can be combined or "concatenated" using the + operator.
    i. In line 8, two literal strings are combined with the string variable to form the output message to the user.
    j. Starting with Python 3.6, a more efficient method for combining strings and variables was introduced. The f-string is used on line 10. Note the f just prior to the literal string value and the curly braces used around the variable name. Also note how the color coding in VS Code assists.
    k. In addition to string, variable can also hold numbers and Boolean values. Boolean values are either True or False.
    l. Lines 12 and 13 create two new Boolean variables named sit and stay. You can change the values from False to True to change the behavior of the code that follows.
    m. In lines 15 through 20, the values of sit and stay are tested using the if statement. Note the use of the colon (:) after the if statement.

© 2020, Keith E. Kelly. All rights reserved.

n. The if statement must contain a condition that evaluates to a Boolean value. In line 15, the value of the sit variable is compared to False. Note the use of the double equal signs.

o. If the condition evaluates to True then the lines of code that are indented after the if are executed.

p. Compare line 12 to line 15 to understand the use of a single equal sign to the use of a double equal sign.

q. In line 12 the right side of the equal sign is being evaluated and "assigned" to the sit variable of the left side. This single equal sign is known as the assignment operator.

r. In line 15 a question is being answered or tested...is the value of sit equal to the value of False – yes or no? The double equal sign is known as a comparison operator and is checking the two values to see if they are equal. The result is a Boolean value (True of False).

s. The if statement requires that the condition that follows evaluates to a Boolean value. In line 22, the code is shortened by recognizing that *sit* is already a Boolean. Note the use of the not operator here to test for a False value.

t. Two logical operators that are used often are "and" and "or". In line 26 the "and" operator is used to test both the sit and stay variables. Do you understand line 28 or are you confused?

u. In lines 31 through 36, the if/elif/else structure is shown. Here a series of values is checked, and the corresponding output is produced. The block always starts with "if" and you can include as many "elif" conditions as necessary.

v. Note that the if and elif statements require a Boolean condition. The else is known as a fall-through. This code block runs if none of the prior conditions evaluate to True. Thus, there is no condition code following else, just the colon.

7. On your own...

a. Create a variable to hold the dog's age. Use the assignment operator (=) to set a value.

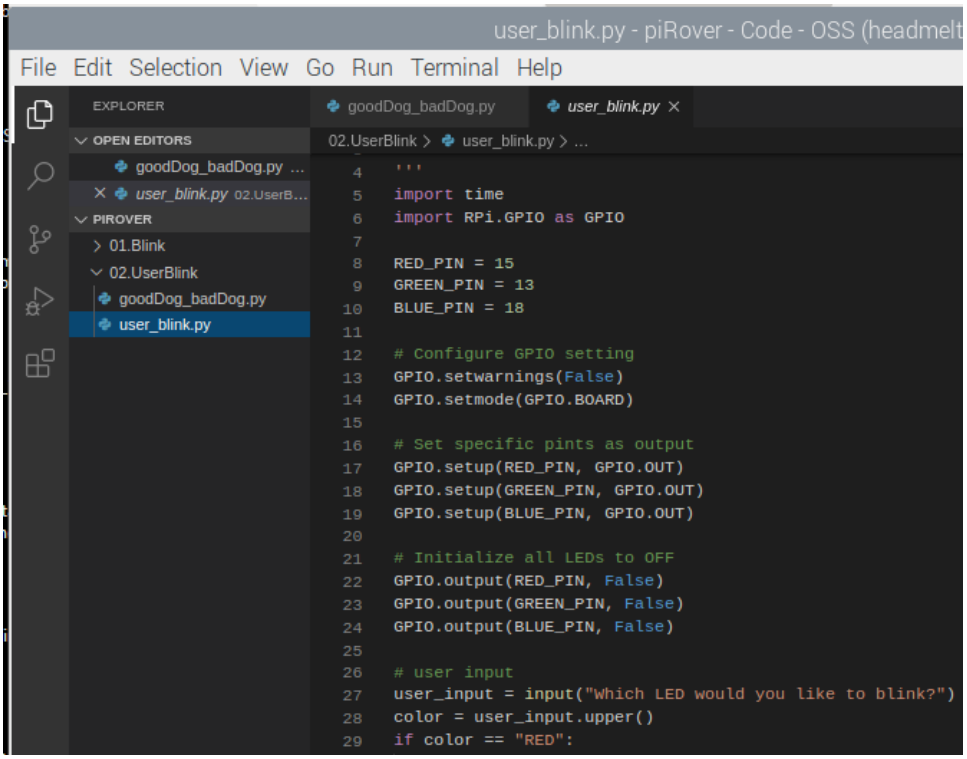
b. Create a new if statement testing the dog's age. If the dog is over 10 years old, then print "Old dog...no new tricks!" Use the greater than (>) comparison operator in your test condition.

© 2020, Keith E. Kelly. All rights reserved.

c. Can you extend this to check if the dog is a puppy? Dogs under 2 are still puppies. Create the required output.

Part 3 – User Blink

8. Open the user_blink.py in the VS Code editor.



9. Run the user_blink.py code. Does it function?

10. Note the pin definitions on lines 8, 9, and 10. These are variables, but the capitalized name indicates that they are "constants", the value will never change during program.

11. Review the GPIO provided on lines 13 through 24. Do you recognize this code from the earlier Blink activity? Note how the pin names are used rather than the literal values of 13, 15, and 18. What if a pin value was changed by the manufacturer? How easy is this code to modify?

12. Could you create the initial GPIO code on lines 13 through 24 on your own?

13. Review the input code on lines 27 and 28. A new function is introduced – upper(). Why is this included? What is the value of adding this to the code?

© 2020, Keith E. Kelly. All rights reserved.

14. The solution functions, but let's refactor or improve the structure. Can you find duplicate code that can be eliminated by introducing a new variable?
15. Enter the revised code shown below. A new pin variable is defined on line 28

```
23  GPIO.output(GREEN_PIN, False)
24  GPIO.output(BLUE_PIN, False)
25
26  # Create a variable for the LED pin
27  # Set its default value to RED
28  pin = RED_PIN
29
30  # ask the user which LED to blink
31  user_input = input("Which LED would you like to blink?")
32  color = user_input.upper()
33  if color == "RED":
34      pin = RED_PIN
35  elif color == "GREEN":
36      pin = GREEN_PIN
37  elif color == "BLUE":
38      pin = BLUE_PIN
39  else:
40      print("Sorry, that is not a valid pin.")
41
42  while True:
43      GPIO.output(pin, True)
44      time.sleep(1)
45      GPIO.output(pin, False)
46      time.sleep(1)
47
```

16. Run the revised solution. Does it function? Is this better code?
17. On your own…
    a. Follow the pattern show for the pin selection to create the ability for the user to specify the blink rate – either fast or slow.
    b. Define a delay variable. If the delay is set to 1 then the blink rate is slow, but if the delay is set to .3 then the blink rate is fast.
    c. Test your code for all combinations of user inputs.
    d. What happens if the user enters an invalid blink speed?
18. Once you completed the solution, copy it to your week folder on your cloud service (OneDrive, Google Drive, or DropBox).

© 2020, Keith E. Kelly. All rights reserved.

piRover Builds with K2

Assessment:

Submit your final **user_blink.py** file to Moodle along with other files in this week's zip file.

© 2020, Keith E. Kelly. All rights reserved.